IBM APPLIED DATA SCIENCE CAPSTONE

Falcon 9 First-Stage Landing Prediction

Will the booster come back? A full data-science walkthrough — REST API and web-scraped data, exploratory analysis with SQL and visuals, geospatial mapping, an interactive dashboard and four competing classifiers.

October 2026 · Space flight data 2010 – 2020

90
Falcon 9 flights analysed

66.7%
booster landing success

0.886
best cross-validated accuracy

github.com/CYL0318/Applied-Data-Science-Capstone

含 AI 生成

AGENDA

From raw API calls to a landing predictor

01 **Executive summary**
What we asked, what we found

02 **Background & problem**
The economics of booster recovery

03 **Data collection — REST API**
SpaceX v4 endpoints via requests

04 **Data collection — web scraping**
Wikipedia launch table via BeautifulSoup

05 **Data wrangling**
Cleaning, labelling, one-hot encoding

06 **EDA — visual analysis**
Six charts, six questions

07 **EDA — SQL analysis**
Ten queries on a SQLite table

08 **Interactive map analysis**
Folium and Plotly Dash

09 **Folium site study**
Markers, records, proximity

10 **Plotly Dash dashboard**
Success by site and by payload

11 **Predictive modelling**
Four classifiers, GridSearchCV

12 **Conclusions & insights**
What the data tells SpaceX

Four answers, one pipeline

90
Falcon 9 launches assembled from API + scrape (2010–2020)

60 / 90
flights that recovered the first stage (66.7%)

0.886
best cross-validated accuracy — Decision Tree

52%
GTO landing rate — the hardest orbit

How we did it

- Collected launch records from the SpaceX v4 REST API and cross-checked them against a Wikipedia scrape.
- Cleaned, labelled (Class 1 = landed) and one-hot encoded the data into 83 features for 90 flights.
- Explored the data with seaborn charts and ten SQL queries, then mapped it with Folium and Plotly Dash.
- Benchmarked Logistic Regression, SVM, Decision Tree and KNN with GridSearchCV (cv = 10).

Key findings

Orbit sets the difficulty. GTO 52% vs VLEO 86% and SSO 100%.
The fleet matured. Yearly success went 0% (2010–13) → 33% (2014–15) → 90% (2019).
Early flights dominate the failures. 22 of 30 failures happened in the first 50 flights.
Model choice is fragile. The Decision Tree wins CV (0.886) but scores 0.778 on hold-out.

Reusability is the whole business case

A Falcon 9 launch is priced at roughly USD 62 million. The first stage — nine Merlin engines, the propellant tanks, grid fins and landing legs — is the most expensive element of the vehicle. Recovering it converts a one-shot expense into an asset that can fly again, which is what makes a faster cadence and a lower price per kilogram possible. When a landing fails, that hardware is lost.

The problem we solve

Given only what is known before launch — site, orbit, payload mass, booster generation, reuse history and grid-fin configuration — can we predict whether the first stage will land? A dependable predictor lets the recovery team position drone ships, price a mission and flag risk before ignition.

MISSION PROFILE

Stage separation
↓
Boost-back / entry burn
↓
Grid-fin guidance
↓
Landing burn
↓
RTLS pad · ASDS drone ship · ocean

Three landing outcomes appear in the data: a return-to-launch-site pad (RTLS), an autonomous drone ship (ASDS) and a miss into the ocean. Only the first two count as a success.

SpaceX REST API: 90 flights from three endpoints

- requests.get on api.spacexdata.com/v4/launches/past returns every historical launch as JSON.
- Each launch only stores IDs, so helper functions follow them into /rockets, /payloads, /launchpads and /cores.
- getBoosterVersion, getLaunchSite, getPayloadData and getCoreData flatten the nested JSON into one row per flight.
- Falcon 1 records are dropped — they carry no first-stage recovery data.
- Output: dataset_part_1.csv with 90 Falcon 9 flights and their landing outcomes.

```python
import requests, pandas as pd

base = "https://api.spacexdata.com/v4"
data = requests.get(base + "/launches/past").json()
cores = {c["id"]: c for c in
         requests.get(base + "/cores").json()}

for launch in data:
    core = cores[launch["cores"][0]["core"]]
    booster = core["serial"]          # B1049
    landing = core["landing_success"] # True/False/None
    pads = launch["launchpad"]        # -> /launchpads
```

GH github.com/CYL0318/Applied-Data-Science-Capstone



The API is the authoritative source for booster serial, landing outcome and coordinates — the fields we later predict.

Web scraping the Wikipedia launch table

1 Request

requests.get(url, timeout=30)
fetches the article HTML

2 Parse

BeautifulSoup(html, "html.parser")
builds a searchable tree

3 Select

soup.find_all("table")
picks the Falcon 9 records table

4 Extract

headers from <th>, rows from <tr>,
citations and footnotes stripped

- The scraped table adds fields the API does not expose directly — the customer, the booster landing text and the launch-site label as written by editors.
- It also serves as an independent cross-check: mismatched rows between API and scrape were reconciled during wrangling.
- Output: spacex_web_scraped.csv, one row per launch with 17 descriptive columns.

GH github.com/CYL0318/Applied-Data-Science-Capstone

```
from bs4 import BeautifulSoup

html = requests.get(url, timeout=30).text
soup = BeautifulSoup(html, "html.parser")
table = soup.find_all("table")[0]
headers = [th.get_text(strip=True)
           for th in table.find_all("th")]
rows = [[td.get_text(strip=True)
         for td in tr.find_all("td")]
        for tr in table.find_all("tr")][1:]
df = pd.DataFrame(rows, columns=headers)
```

From nested JSON to one clean modelling table

01 Filter

Keep Falcon 9 only. Falcon 1 rows carry no booster data and would inject nulls into the target.

02 Impute

A handful of flights have no PayloadMass; those are filled with the column mean so the numeric pipeline stays complete.

03 Label

Class = 1 for a landed booster (True RTLS, True ASDS); Class = 0 for False RTLS, False ASDS, False Ocean and None outcomes → 60 vs 30.

04 Encode

One-hot encode Orbit, LaunchSite, BoosterVersion, GridFins, Reused and Legs, giving 83 binary/numeric features for 90 rows.

Landing outcome → Class

Outcome field	Class
True RTLS	1
True ASDS	1
False RTLS / ASDS / Ocean	0
None ASDS / None None	0

90 rows × 83 features

No missing values in the modelling table.
Target balanced 60 / 30 — the classes are not skewed.

GH github.com/CYL0318/Applied-Data-Science-Capstone

Six charts, six questions

Flight number vs launch site

Does experience at a pad matter? Colour = landing class.

Payload mass vs launch site

Are heavy payloads restricted to certain pads?

Mean success rate by orbit

Which orbits are hardest to land from?

Flight number vs orbit

How did the orbit mix change over the programme?

Payload mass vs orbit

Do heavy GTO payloads fail more often?

Yearly success rate

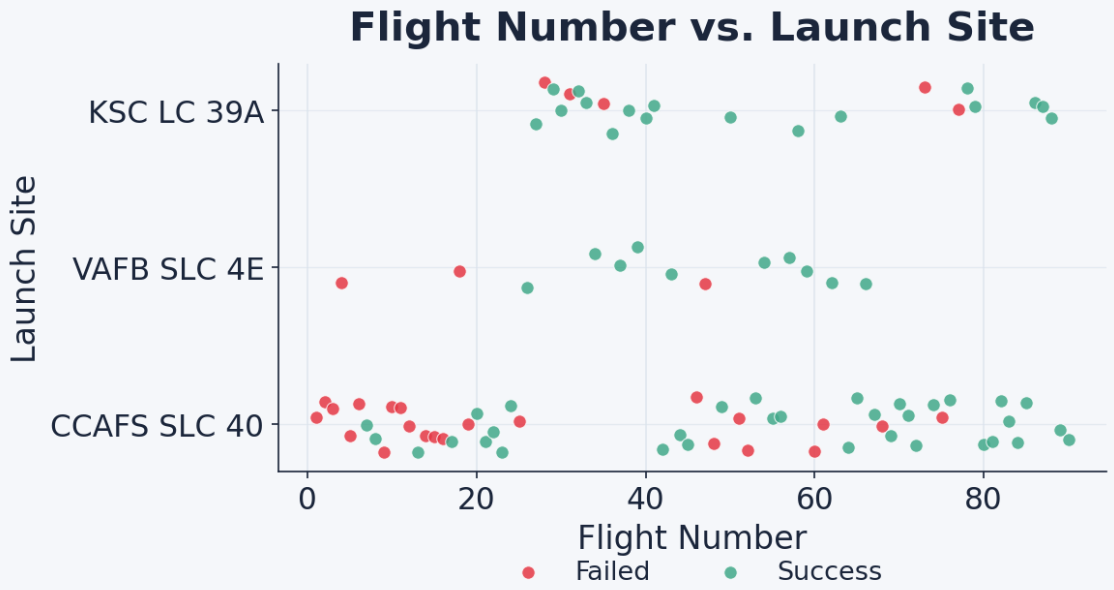
Is reliability improving as the fleet matures?

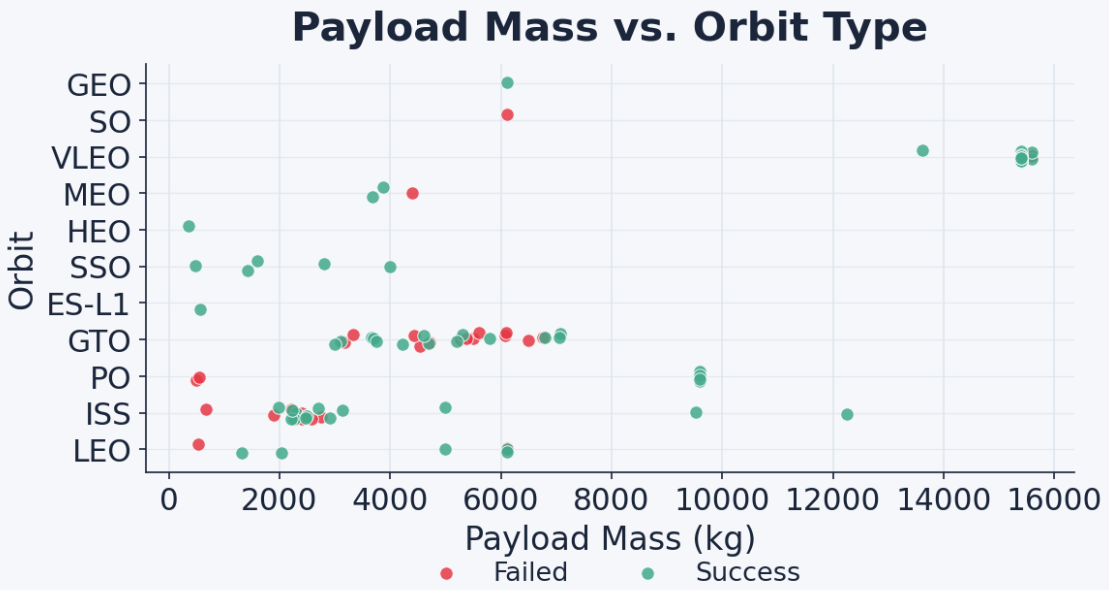
Every chart below is produced by the seaborn/matplotlib notebook jupyter-labs-eda-dataviz.ipynb; each one is followed in the notebook by a written observation.

Charts use landing class as hue so failures are visible as a second series rather than buried in an average.

GH github.com/CYL0318/Applied-Data-Science-Capstone

Failures concentrate in the early flights

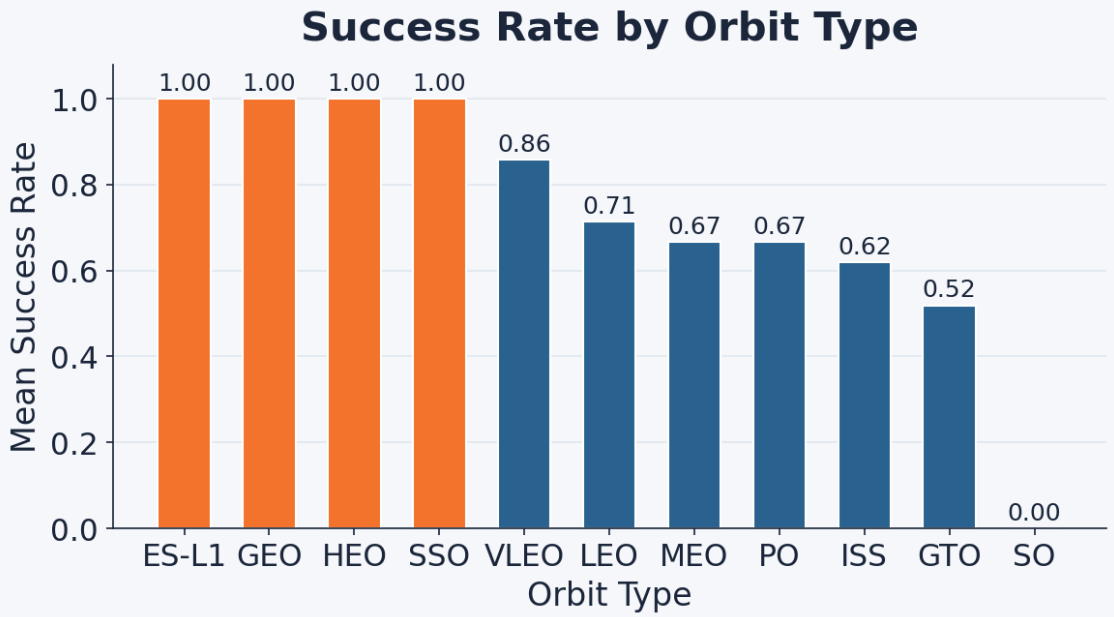


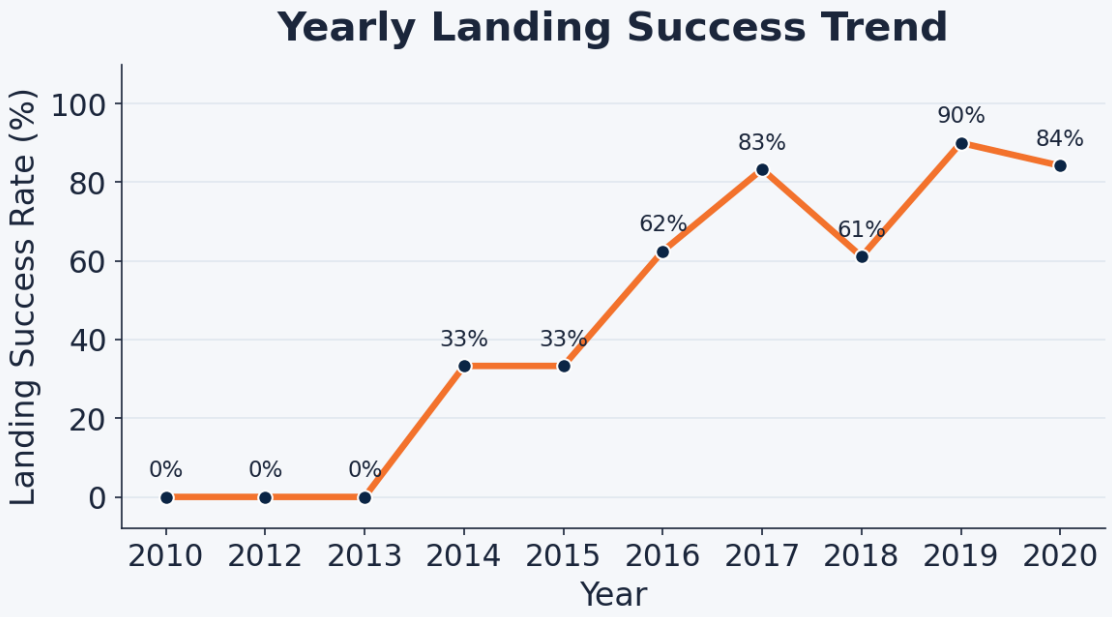


Failures (red) sit almost entirely before flight 70: 22 of the 30 failures fall inside the first 50 flights, and the final 20 flights land 85% of the time.

No payload-mass cut-off separates the classes — heavy GTO payloads fail more often, but light payloads fail too. Orbit type is the clearer discriminator.

GTO is the weak spot; the fleet has matured





SSO, ES-L1, GEO and HEO sit at 100% on small samples; GTO at 52% (27 flights) is the only large, low-performing bucket.

Yearly success climbs 0% (2010–13) → 33% (2014–15) → 83% (2017) → 90% (2019) → 84% (2020), with a dip to 61% in 2018.

Ten queries on a SQLite launch table

- The scraped CSV is loaded into a SQLite table, SPACEXTBL, and queried with %sql inside the notebook.
- The ten queries answer: which sites exist, how many CCAFS records there are, total and average payload, first successful ground-pad landing, booster versions with drone-ship success, overall outcome counts, boosters carrying maximum payload, failed drone-ship landings by month, and a ranked landing-outcome table.
- SQL is used where an aggregate question is faster to state than to write in pandas — counts, averages and rankings.

> *Three families of question: how many (counts by site and by year), how well (success rate by orbit and booster version), and when (failed drone-ship landings by month).*

GH github.com/CYL0318/Applied-Data-Science-Capstone

> *Query results feed the two result slides that follow, so the numbers on them are reproducible from the notebook.*

```
SELECT DISTINCT LaunchSite FROM SPACEXTBL;

SELECT Orbit,
       ROUND(AVG(Class)*100, 1) AS success_pct,
       COUNT(*) AS n
FROM SPACEXTBL
GROUP BY Orbit
ORDER BY success_pct DESC, n DESC;

SELECT substr(Date,1,4) AS yr, COUNT(*) AS launches
FROM SPACEXTBL GROUP BY yr ORDER BY yr;

SELECT Class, COUNT(*) AS n FROM SPACEXTBL
GROUP BY Class ORDER BY n DESC;
```

Site, payload and orbit tell most of the story

Launches by site

Launch site	Launches	Success	Payload kg
CCAFS SLC 40	55	60.0%	305,151
KSC LC 39A	22	77.3%	167,342
VAFB SLC 4E	13	76.9%	76,953
Total	90	66.7%	549,446

CCAFS SLC 40 flew 55 missions but only 60% landed — it absorbed almost every early-era failure. KSC LC-39A and VAFB, which only entered service later, sit near 77%.

The payload column is a sum across every flight from that pad, not a per-flight average — which is why the total row is the sum of the three sites.

Success rate by orbit

Orbit	Success	Flights
SSO / GEO / HEO / ES-L1	100%	8
VLEO	86%	14
LEO	71%	7
PO	67%	9
MEO	67%	3
ISS	62%	21
GTO	52%	27

GTO carries the highest energy demand: the booster has the least propellant left for a landing burn, and it is the orbit with the most flights, so the 52% is not a small-sample artefact.

Outcome ranking and cadence over time

Outcome ranking

Class	Meaning	Flights	Share
1	Landed	60	66.7%
0	Not landed	30	33.3%

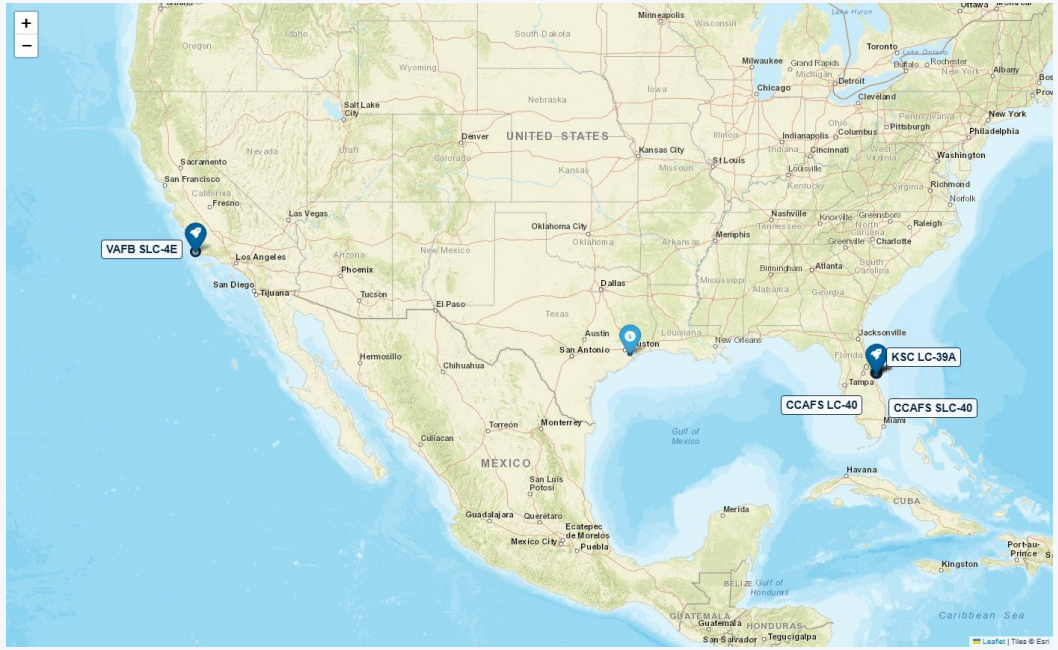
A 60 / 30 split means a naive majority-class guess already scores 66.7% — any model must be judged against that baseline, not against zero.

Cadence and success by year

Year	Launches	Success
2010	1	0%
2012	1	0%
2013	3	0%
2014	6	33%
2015	6	33%
2016	8	63%
2017	18	83%
2018	18	61%
2019	10	90%
2020	19	84%

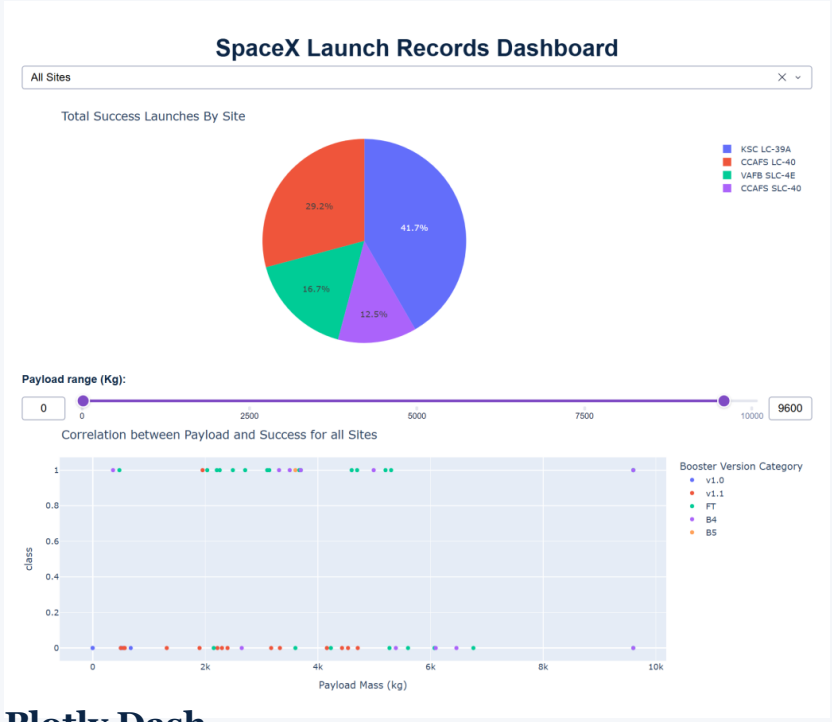
Cadence rose faster than reliability slipped: launches tripled from 2016 to 2017 while the success rate still climbed, so reuse scaled without eroding the learning curve.

Two interactive tools: a map and a dashboard



Folium

Launch sites and individual flights plotted on real coordinates, colour-coded by landing outcome, with distance lines to nearby infrastructure.

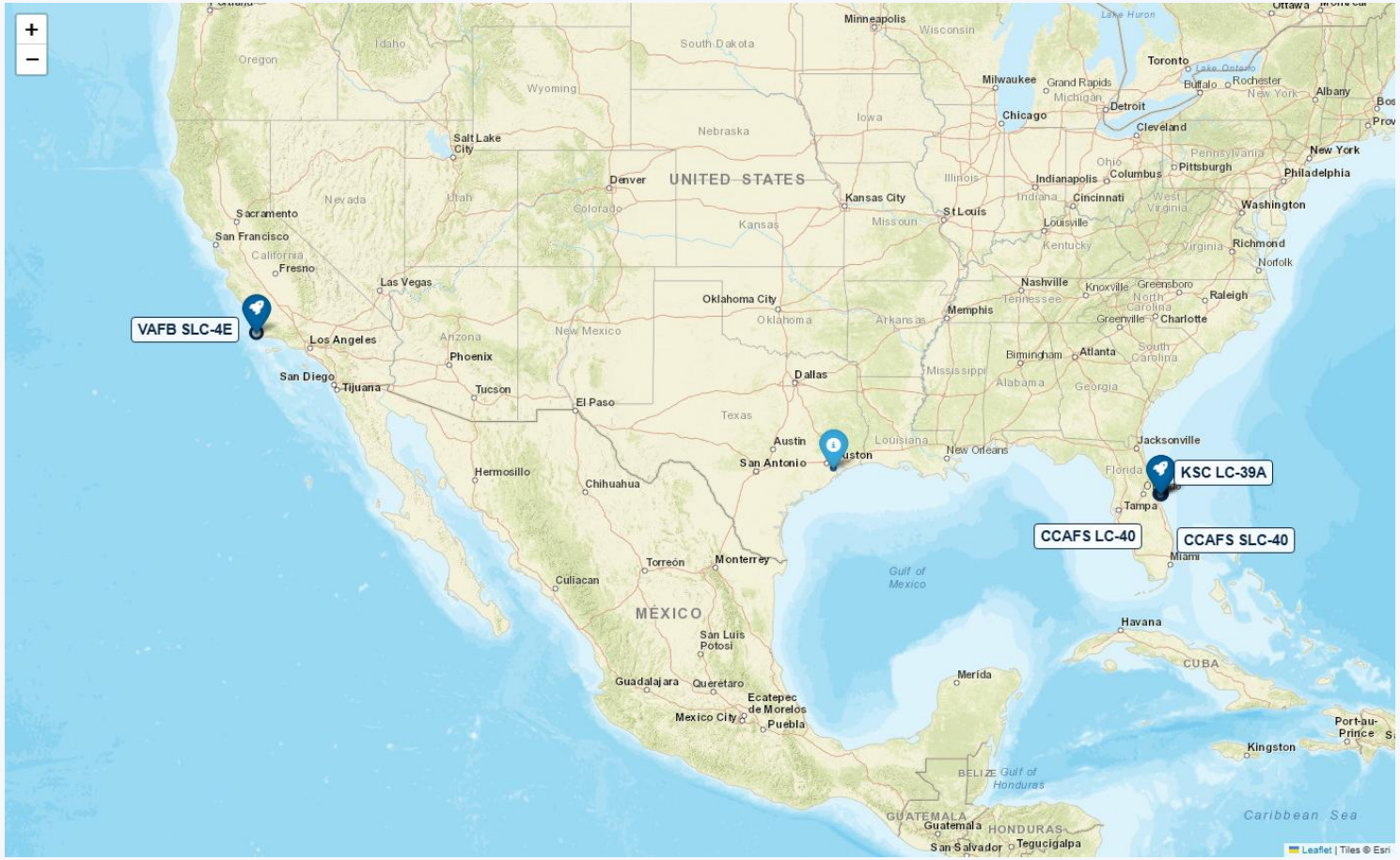


Plotly Dash

Dropdown by site, success pie, payload range slider and a payload-vs-outcome scatter, all redrawn by callbacks.

Repository: github.com/CYL0318/Applied-Data-Science-Capstone

All four pads sit on the coast, on two oceans

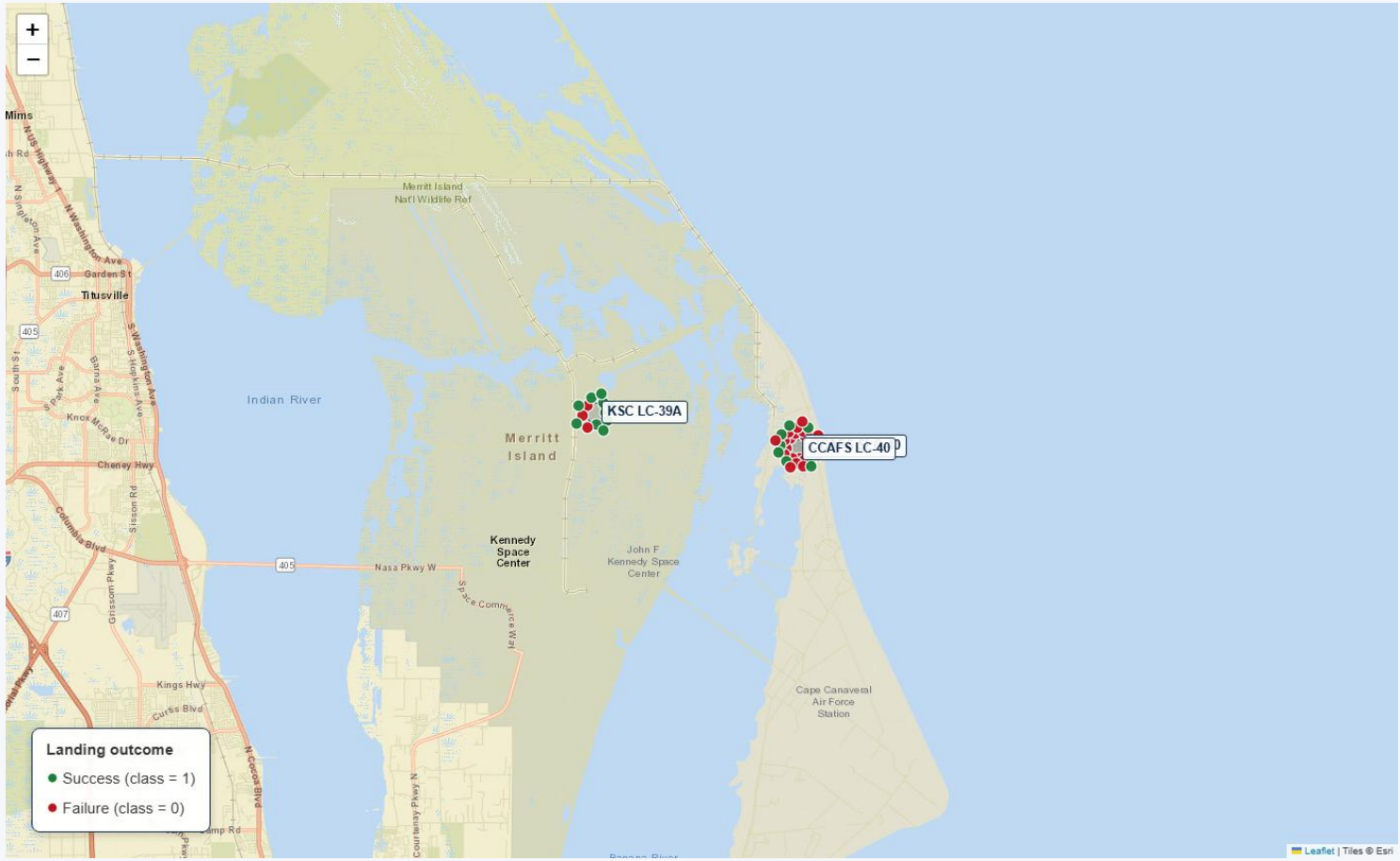


What the map exposes

- CCAFS LC-40 and CCAFS SLC-40: adjacent pads on Florida's Atlantic coast.
- KSC LC-39A: inside Kennedy Space Center, 13 km north of the CCAFS pads.
- VAFB SLC-4E: California's Pacific coast, used for polar and SSO missions.
- Every site is within a few kilometres of the ocean — a failed landing ends in water, not on land.

Markers are drawn with folium.Marker and a Circle of coverage; the NASA Johnson Space Center reference point from the lab is included for orientation.

Every flight, colour-coded by landing outcome

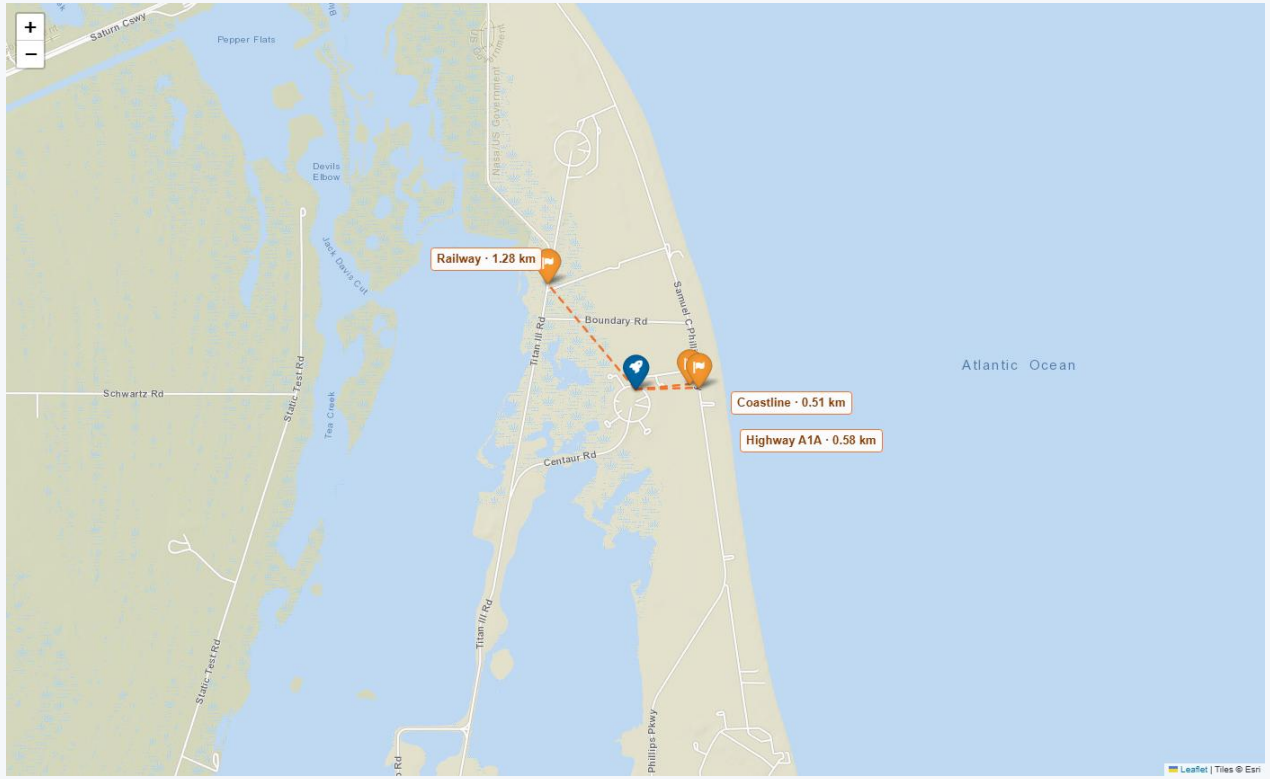


Reading the pattern

- One marker per flight, green for a landed booster, red for a failure.
- The Cape Canaveral cluster is visibly mixed — it carries the early programme.
- KSC LC-39A's ring is almost entirely green: it only hosted later, mature missions.
- VAFB SLC-4E is off-frame at this zoom; its 13 flights are shown on the overview map.

Markers are jittered around each pad with a golden-angle spiral so overlapping flights stay countable — otherwise 55 CCAFS records collapse into one dot.

What sits next to a launch pad?



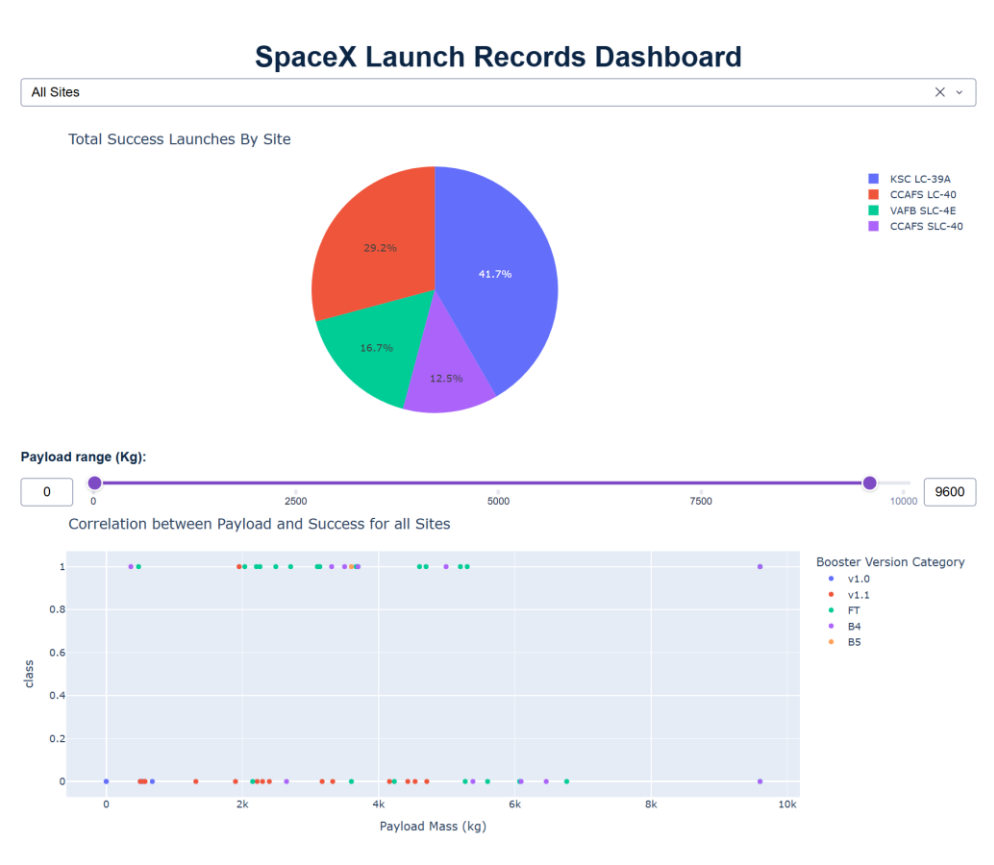
Distance from CCAFS SLC-40

Feature	Distance
Coastline	0.51 km
Highway A1A	0.58 km
Railway	1.28 km
Melbourne (city)	53.42 km

Distances computed with the haversine formula and drawn as folium.PolyLine segments between the pad and each flagged point.

> *The pattern is deliberate: pads hug the coast (over-water abort paths), sit beside a highway and railway (oversized payload transport) and stay far from cities (acoustic and debris safety).*

Dashboard: success share and payload correlation

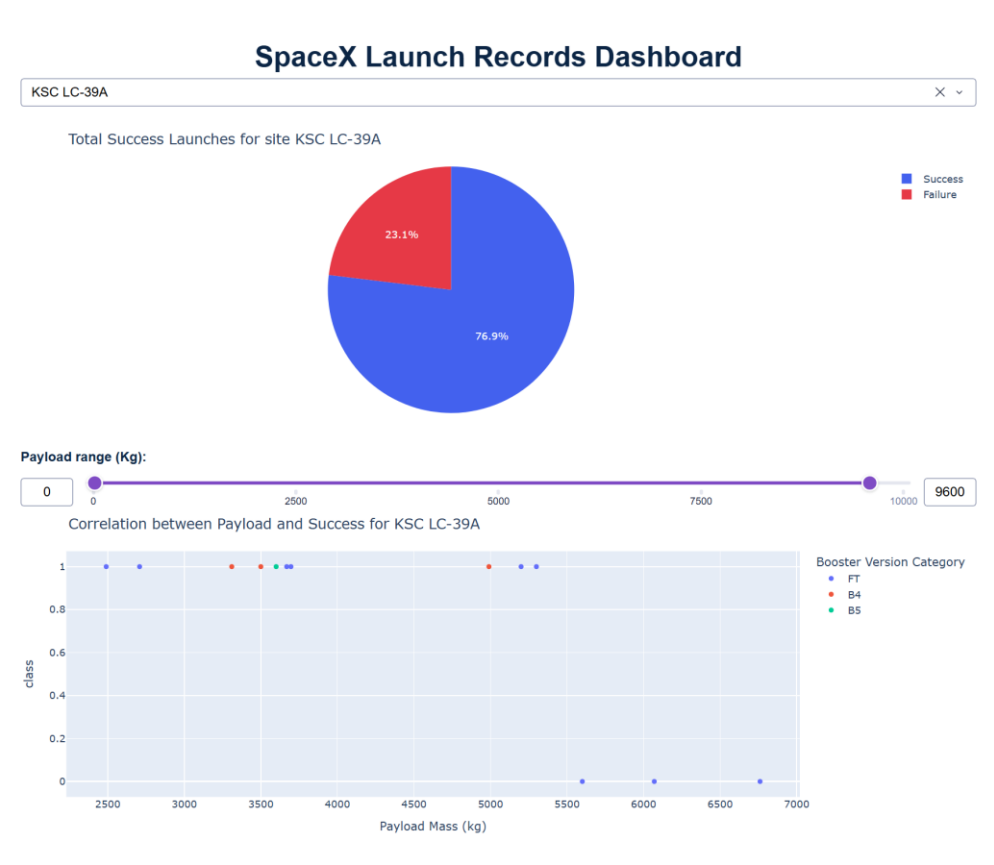


What the controls do

- Site dropdown — All Sites, or one of the four pads.
- Pie chart — share of successful launches per site.
- Payload range slider — 0 to 10,000 kg, 1,000 kg steps.
- Scatter chart — payload mass vs outcome, coloured by booster version.

With all sites selected, KSC LC-39A contributes 41.7% of all successful landings in the dashboard dataset, from only 13 of its 56 launches.

Drill-down: one pad, one clean split



Filtered view — KSC LC-39A

- The pie flips from site share to a success/failure split: 76.9% vs 23.1%.
- The scatter only plots that pad's missions, so the payload window tightens to roughly 2,000–6,700 kg.
- Booster generation separates clearly — FT and B5 missions land, early v1.0/v1.1 missions do not.

Callbacks re-run on every dropdown or slider change, so the two charts always describe the same filtered slice of the data.

Four classifiers on one honest benchmark

Logistic Regression

C tuned over 0.01–10, l2 penalty,
lbfgs solver

Support Vector Machine

linear / rbf / poly / sigmoid kernels,
C and gamma over 7 values each

Decision Tree

gini vs entropy, depth 2–12,
max features, leaf and split size

K Nearest Neighbors

k = 1–10, four algorithms,
Manhattan and Euclidean distance

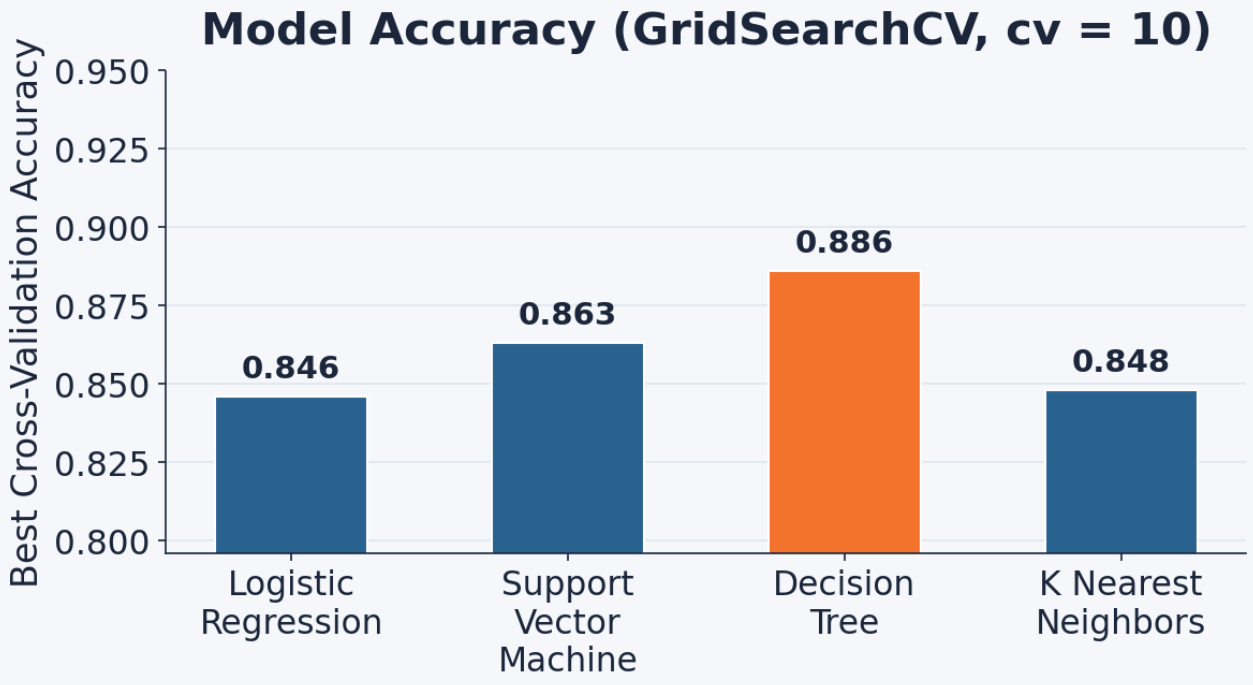


Why this setup

- Features are scaled so KNN and SVM are not dominated by payload mass.
- Hyper-parameters are chosen inside a 10-fold CV on the training split only — the 18 hold-out flights are never seen during tuning.
- The hold-out set is fixed (random_state = 2) so all four models are compared on the same 18 flights.

Caveat
18 test flights is a small sample: one flight moves accuracy by 5.6 points. Test scores are therefore a sanity check, not a ranking.

Decision Tree wins cross-validation — narrowly

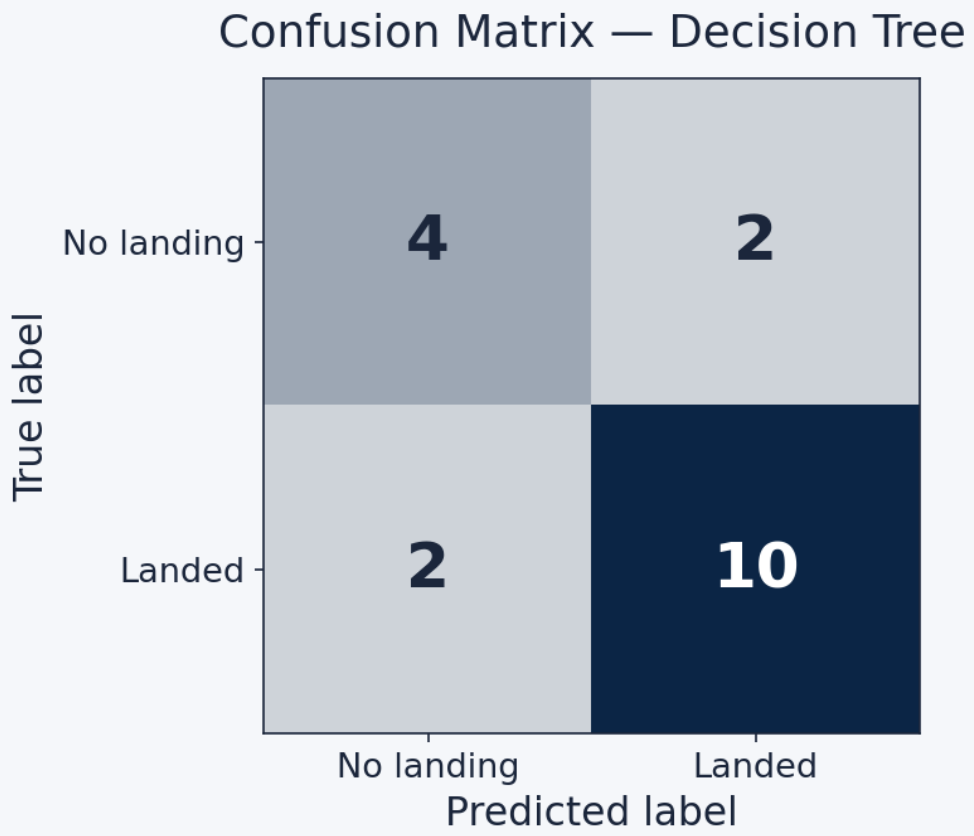


Model	Best CV	Hold-out
Logistic Regression	0.846	0.833
Support Vector Machine	0.863	0.833
Decision Tree	0.886	0.778
K Nearest Neighbors	0.848	0.833

The tree has the best CV score but the weakest hold-out score — a sign that its splits are fitting quirks of the small training set.

All four models beat the 66.7% majority baseline, but Logistic Regression and KNN trade a little CV accuracy for a more stable hold-out result.

It catches 10 of 12 landings, and 2 of 6 misses



Reading the matrix

10 true landings correctly predicted (recall 83%).

4 true non-landings correctly rejected.

2 landings predicted as failures and **2** failures predicted as landings.

Accuracy 0.778 · precision 0.83 · recall 0.83.

What the errors mean in practice

- A false landing prediction sends a drone ship to a mission that will not need it — a wasted recovery asset.
- A false failure prediction discourages reuse planning for a booster that would have returned safely.
- With 18 test flights, either error costs 2 flights — so the model is a prioritisation aid, not an autonomous decision-maker.

GH github.com/CYL0318/Applied-Data-Science-Capstone

What the data actually tells SpaceX

01 **Orbit is the difficulty dial**

GTO lands 52% of the time while VLEO lands 86% and SSO 100%. High-energy missions leave the least propellant for the landing burn.

02 **Generation beats geography**

Booster generation separates outcomes far more cleanly than launch site: post-Block-4 boosters carry the 2019–2020 84–90% success.

03 **Experience compounds**

22 of 30 failures fall in the first 50 flights; the last 20 flights land 85% of the time. Site service age, not location, explains CCAFS's low rate.

04 **Payload alone is a weak signal**

No payload-mass threshold separates the classes — payload matters chiefly because it correlates with orbit.

A PRACTICAL USE

Flag high-risk recoveries before launch

Combine the three strongest signals into a simple pre-launch rule:

GTO orbit
+ payload above 5,500 kg
+ a first-flight booster

→ schedule the drone ship with a recovery margin and expect a miss in the ocean.

In the data, payloads above 5,500 kg succeeded 30% of the time against 55% for the 2,000–5,500 kg band.

CONCLUSION

Landing success is predictable, but only within a narrow margin

Ninety flights, two data sources, ten SQL queries, four maps, one dashboard and four classifiers point to the same conclusion: orbit and booster generation carry most of the signal, experience carries the rest, and the choice of model matters less than the data it learns from.

Thank you

Full notebooks, data and the Dash app:

github.com/CYL0318/Applied-Data-Science-Capstone